1 · Unit Testing with Jest

Exam cheat sheet · [+] = added from outside docs, not in slides

Bugs & error types

- **Bug** = defect in a program that makes it behave wrongly. **Debugging** = fixing it. (Name from a moth stuck in a relay of the Mark II computer.)
- **Syntax error**: invalid code typed (missing quote, `if age === 20 {`, reserved word `class` as a name). Caught before running.
- **Run-time error**: fails *while executing*. e.g. `"Free".toFixed(2)` → string has no `toFixed`.
- **Logic error**: runs but gives the wrong result. e.g. `math + science / 2` (precedence) instead of `(math + science) / 2`.

Software testing

- Goal: check the software meets requirements and find defects.
- Done by: tester, developer, project lead/manager, end user.
- **Types**: Unit → Integration → System → UAT (works for users) ; **Regression** = checks for unexpected side effects of changes.
- **Test pyramid**: Unit (bottom, isolated, fast) → Component → Integration → API → UI (top, integrated, slow).
- **Non-functional**: performance, volume, security, reliability, availability, reusability, flexibility.
- **Unit test**: tests one small unit (function/class) independently; usually by developers, during development; pass/fail. Examples: `validatePassword()`, `calculateDiscount()`, `isPalindrome()`.

	Manual	Automated
Pros/cons	boring, human error, slow (2-3k cases!)	reliable, fast, repeatable
Best for	exploratory, usability, ad-hoc	highly repeatable functional/non-functional

TDD: write the test first — **Red** (fail) → **Green** (pass) → **Refactor**.

Semantic Versioning X.Y.Z: **Major** = breaking API change; **Minor** = new features, backward compatible; **Patch** = bug fixes.

Frameworks: **Jest**, Vitest, Node native test runner, Mocha.

Test case design

Ask: what should work? what must be rejected? boundaries (min/max)? each branch (if/else)? can it throw?

- **Positive test**: input meets the rules → expect accepted.
- **Negative test**: input breaks the rules (on purpose) → expect rejected/error. e.g. password rule ≥ 8: use 3 chars; ATM max 20,000: try 25,000.
- **Boundary value**: test at the edge and either side. Rule ≥ 8: `7→false, 8→true, 9→true`. Most bugs hide at `< <= > >=`. (`length > 8` vs `length >= 8` differ only at 8.)
- Negative inputs: `null`, `12345` (number), `""`, `"john doe"` → false.

Username example (5-20 chars; letters/digits/_ only; no spaces; starts with letter)
Pos: "john1" (min 5), "john_doe", "User_123", "abcde12345", 20 chars (max).
Neg: "john" (4), 21 chars, "1john", "john doe", "john@123".

Setup & syntax

```
npm init -y
npm install -D jest
// package.json:  "scripts": { "test": "jest" }
npm test
```

```
// sum.js
function sum(a, b) { return a + b; }
module.exports = sum;

// sum.test.js
const sum = require("./sum");
test("2 + 3 = 5", () => {
  expect(sum(2, 3)).toBe(5);
});
```

- `test(name, fn)` / `it(...)` = one **test case**. File `*.test.js` = **test suite**. Output: "Test Suites: 3 passed" (files) vs "Tests: 3 passed" (cases).
- `describe(name, () => {...})` groups related tests; several per file; `it("should ...")` reads like English.
- `expect(actual).matcher(expected)`: "I expect add(2,3) to be 5". `expect(...)` + matcher = an **assertion**.

AAA pattern: **Arrange** (prepare data) → **Act** (call function) → **Assert** (check result)

```
test("member gets 10% discount", () => {
  const price = 1000, member = true;      // Arrange
  const result = calculateDiscount(price, member); // Act
  expect(result).toBe(900);               // Assert
});
```

Matchers

Matcher	Use
`toBe(x)`	primitives, exact (`Object.is`); also `null`, `undefined`, true/false, strings
`toEqual(x)`	**objects/arrays** — recursively compares fields (`[]`, `{}`, nested)
`toBeTruthy()` / `toBeFalsy()`	Falsy: `false, 0, "", null, undefined, NaN`. Everything else truthy (negative numbers, objects, `[]`, string "false")
`toContain(x)`	item in array, or substring in string
`toThrow(msg|Error)`	wrap call in a function: `expect(() => divide(10,0)).toThrow("Cannot divide by zero")`
`.not`, `toStrictEqual`, `toBeCloseTo` [+]	negate; stricter equality (checks undefined keys/types); floats (0.1+0.2)

Quiz answers from slides: unit test = test small unit independently; objects → `toEqual`; negative test = invalid input (3-char password, 25,000 withdrawal).

Project work: summarize tests for ≥ 5 features, e.g. `validateUsername()` (valid / invalid char / length boundary), `canAccessAdmin()` (admin / normal user).

2 · API Testing

What & where

- API testing sits between integration and UI in the test pyramid: more integrated & slower than unit tests, faster/more stable than UI tests.
- An API call = send an **HTTP request** (method + URL + headers + body) → check the **response** (status code, headers, body).
- **API types**: **REST**, **SOAP** (XML), **GraphQL**.
- Functions/APIs are reusable blocks (input → function → output); avoids **spaghetti code** (unstructured, hard to maintain).

Calling APIs

- The **Accept** header chooses the format: `text/html` (default), `application/json`, `text/plain`. curl without Accept → `text/plain` (on icanhazdadjoke.com).

```
curl -H "Accept: application/json"
https://icanhazdadjoke.com/
curl https://randomuser.me/api/
```

- Tools: **curl** (CLI), **Postman** (desktop or web.postman.co — no install), browser **DevTools** (copy request), VS Code REST Client, `<url>/docs` (API documentation page).

HTTP methods (CRUD)

Method	Action	Typical success
GET	read	200 OK
POST	create	201 Created
PUT	update/replace	200 OK
DELETE	delete	200 / 204 [+]

Status codes [+]

Code	Meaning
200 / 201	OK / Created
400	Bad request (invalid input)
401	**Unauthorized** = not authenticated (no/invalid credentials) — "who are you?"
403	**Forbidden** = authenticated but not allowed — "what can you do?"
404	Not found
500	Server error — usually a **bug** (assignment: try to provoke 500s)

Secure APIs

- **Authentication** = who are you? **Authorization** = what may you do?
- Flow in the lab: **Login** (username/password) → receive **token** → copy token → **Authorize** / send it with each request.
- **Basic Auth**: header = `Authorization: Basic base64(username:password)`. **Base64 is encoding, NOT encryption** (reversible, e.g. base64decode.org) → use only over **HTTPS**.
- Lab users: user1, user2, anonymous (password 12345).

Lab: To-do list API (what was tested)

Call	URL / what to check
GET	`/tasks/1`, `/user`, `/user/user1`, `/user/me`
GET + query	`/tasks?status=In%20Progress`, `/tasks?search=Keep` (query string params; `%20` = space)
POST	create task (as user1), then GET to verify it exists
PUT	`/tasks/1` edit text / status Draft → In Progress. Body may include id, but the **ID from the URL is used** (changing 1→100 in the body does nothing)
DELETE	`/tasks/1` — without auth → denied

- **Bug found**: PUT with an empty body **deleted the email** (should validate/reject).
- **Correct behaviour**: logged in as user1 → cannot modify other users' data; user2 → cannot delete user1's task (authorization works).
- An endpoint with "no documentation" still has to be tested (discover via DevTools / `/docs`).

API test checklist [+]

- **Status code** correct? **Body** (JSON fields/values)? **Headers** (Content-Type)?
- **Positive** (valid input) & **negative** (missing/invalid fields, wrong types, empty body).
- **Boundaries** (min/max lengths, ids that don't exist).
- **Auth**: no token (401), wrong role/other user's data (403), valid token (200).
- **Side effects**: after POST/PUT/DELETE, GET again to confirm state.

> **Remember**: 401 = *authentication* problem (no/invalid login). 403 = *authorization* problem (logged in, no permission). Base64 ≠ encryption. 500 on bad input = bug.

3 · Model-View-Controller (MVC)

Concept

- **MVC** = software architecture pattern splitting an app into **3 parts** with separate duties → tidy code, easy to change, easy teamwork. Benefit grows as functions grow.
- Based on OOP: three objects that **must not interfere with each other's job**.
- MVC frameworks: Express, Flask, Django, Laravel, Rails, CodeIgniter.

Part	Responsibility
Model	All **data** & DB access; prepares data; works only when the Controller asks
View	**UI** / what the user sees; receives commands from Controller; passes user actions to it
Controller	The "brain": all **logic**; middleman between Model & View; everything waits for its orders

Login example: User → **V** → **C** → **M** (asks DB to verify) → result back to **C** →
success: C tells V to show main page | fail: C tells V to show "wrong password".

Express project structure (lab)

```
project/
 configs/db.js            // DB connection (mysql2 +
dotenv)
 routes/productRoutes.js  // URL -> controller function
 controllers/productController.js  // req/res + logic
 models/productModel.js   // SQL queries
 views/products.ejs       // HTML templates (EJS)
 public/css/              // static files
 .env  index.js  package.json
```

- **Route** = maps URL+method to a controller function. **Controller** = reads `req`, calls Model, sends `res` (JSON or rendered view). **Model** = `db.query(sql, params, callback)`.
- **Refactor order in lab**: split config → routes (`app` → `express.Router()`, `module.exports = router`) → controller → model.

```
// index.js
const productRoutes = require('./routes/productRoutes');
app.use(express.json());
app.use('/api', productRoutes);      // all routes get
/api prefix
app.listen(PORT);
```

```
// routes/productRoutes.js
const router = express.Router();
const productController =
require('../controllers/productController');
router.get('/products',
productController.getAllProducts);
router.get('/products/:id',
productController.getProductById);
router.get('/products/search/:keyword',
productController.searchProducts);
router.post('/products',
productController.createProduct);
router.put('/products/:id',
productController.updateProduct);
router.delete('/products/:id',
productController.softDeleteProduct);
router.put('/products/restore/:id',
productController.restoreProduct);
module.exports = router;
```

```
// controllers/productController.js
const Product = require('../models/productModel');
const productController = {
  getAllProducts: (req, res) => {
    Product.getAll((err, results) => {
      if (err) return res.status(500).json({ error:
err.message });
      res.json(results);
    });
  },
};
module.exports = productController;
```

```
// models/productModel.js
const db = require('../configs/db');
const { v4: uuidv4 } = require('uuid');   // new id on
create
const Product = {
  getAll: (cb) => db.query(
    'SELECT * FROM products WHERE is_deleted = 0', cb),
  getById: (id, cb) => db.query(
    'SELECT * FROM products WHERE id = ? AND is_deleted =
0', [id], cb),
};
module.exports = Product;
```

- **Soft delete**: `UPDATE ... SET is_deleted = 1` (row kept, hidden by `WHERE is_deleted = 0`); **restore** sets it back to 0.
- `?` placeholders + params array = parameterized query (prevents SQL injection [+]).
- Search uses `LIKE ?` with `'%keyword%'`.

View & EJS template engine

- **View** = presentation layer: dynamic HTML built from data the controller sends. **Template engine** (EJS, Pug) inserts data into HTML. Install: `npm i ejs`; `app.set('view engine','ejs')` [+].
- Controller renders: `res.render('products', { products: results })` → `views/products.ejs`.
- **Static files** (CSS): `app.use(express.static('public'))`, then `<link rel="stylesheet" href="/css/style.css">`.

EJS tag	Meaning
`<%= name %>`	print value, **HTML-escaped** (safe from XSS)
`<%- html %>`	print **raw** HTML (unescaped)
`<% code %>`	run JavaScript, no output (if / forEach)
`<%- include('partials/header') %>`	insert another .ejs file

```
<% products.forEach(function(p) { %>
  <li><%= p.name %> - <%= p.price %></li>
<% }) %>
<% if (age >= 18) { %> adult <% } else { %> minor <% } %>
```

Remember: Route → Controller → Model → DB, then Controller → View. Model never touches the UI; View has no business logic; Controller holds the logic.

4 · Cookies & Sessions (+ password hashing)

Exam cheat sheet · [+] = added from outside docs, not in slides · JWT = not examined

What is a cookie?

- Small data the **browser stores** to remember state (e.g. **Session ID**). Server sets it with `Set-Cookie` (response); browser sends matching cookies back in the `Cookie` header of later requests ⊕.
- **Uses**: authentication (stay logged in), session management, personalization (language/theme), shopping cart, analytics, advertising.
- **Never store in cookies**: passwords (plain or hash), credit card/CVV, sensitive personal data (ID no., health), private keys/API secrets, DB credentials, **roles/permissions** (e.g. `isAdmin=true`), confidential business data.
- **Cookie vs session**: cookie holds only the **session ID**; the real user data stays on the **server** (`SESSIONS` Map).

Cookie attributes

Attribute	Effect
`httpOnly: true`	JavaScript can't read it (`document.cookie`) → protects from XSS theft
`secure: true`	sent only over **HTTPS** (localhost exempt)
`sameSite`	`"none"` = sent on cross-site requests (needs Secure ⊕); `"lax"` / `"strict"` limit cross-site sending (CSRF protection). Chrome/Edge treat unset as Lax ⊕
Expires / Max-Age, Domain, Path ⊕	lifetime (none = session cookie) and which URLs receive it

Project I — session-cookie login

Install: `npm i express cors cookie-parser`. Server `http://localhost:3000`, client served by Live Server `http://127.0.0.1:5500` (different origin = cross-origin).

```
// server.js
app.use(express.json());
app.use(cors({ origin: "http://127.0.0.1:5500",
credentials: true }));
app.use(cookieParser());
const USERS = new Map();    // username -> { id,
username, role }
const SESSIONS = new Map(); // sessionId -> user

app.post("/login", (req, res) => {
  const user = USERS.get(req.body.username);
  if (user == null) { res.sendStatus(401); return; }
  const sessionId = crypto.randomUUID();     // UUID v4
  SESSIONS.set(sessionId, user);             // store on
server
  res.cookie("sessionId", sessionId, {
    secure: true, httpOnly: true, sameSite: "none" })
     .send(`Authed as ${req.body.username}`);
});

app.get("/adminData", (req, res) => {
  const user = SESSIONS.get(req.cookies.sessionId); //
read cookie
  if (user == null) { res.sendStatus(401); return; } //
not logged in
  if (user.role !== "Admin") { res.sendStatus(403);
return; } // no permission
  res.send("Authorized: You are viewing restricted
administrator controls.");
});
```

```
// client.js
fetch("http://localhost:3000/adminData", {
  credentials: "include",   // REQUIRED to send cookie
cross-origin
  headers: { "Content-Type": "application/json" } })
```

- **Without** `credentials:"include"` the request has no `Cookie` header → 401. **With** it: `Cookie: sessionId=xxxx`. Server must also allow `credentials: true` in CORS.
- **401** = not logged in / invalid session. **403** = logged in but role not allowed (Student calling admin data).
- **Bug fix**: when a login fails, don't keep the old session — `SESSIONS.delete(req.cookies.sessionId)` + `res.clearCookie("sessionId", {secure:true, httpOnly:true, sameSite:"none"})` then `401`. (Otherwise a Student whose login failed could still hold an Admin session.)
- DevTools → Application → Cookies shows `sessionId` and its flags.
- **Demo**: the-internet.herokuapp.com (tomsmith / SuperSecretPassword!). Copy the `rack.session` value, paste into an Incognito window → you're logged in (session hijack idea). Wrong value → redirect to /login.

Project II — storing passwords (bcrypt)

Storage	Risk
Plain text	anyone reading DB sees passwords
Encrypted	reversible — leaked key = all passwords
Hash	one-way; **cannot be reversed** → best

- `bcrypt.hash(password, 10)`: **10 = cost factor** (higher = slower = harder to brute-force; it is not the salt and not the length). Output is **60 chars**, e.g. `$2b$10$...`
- **Salt** = random value added before hashing → same password gives **different hashes** each time (`hash1 === hash2` → false).
- `bcrypt.compare(password, hash)` → `true` for any of those hashes, because the salt & cost are **stored inside the hash**.

```
app.post("/users", async (req, res) => {
  try {
    const salt = await bcrypt.genSalt();
    const hashed = await bcrypt.hash(req.body.password,
salt);
    users.push({ name: req.body.name, password: hashed
});
    res.status(201).send();
  } catch { res.status(500).send(); }
});
app.post("/users/login", async (req, res) => {
  const user = users.find(u => u.name === req.body.name);
  if (user == null) return res.status(400).send("Cannot
find user");
  try {
    if (await bcrypt.compare(req.body.password,
user.password))
      res.send("Success"); else res.send("Not Allowed");
  } catch { res.status(500).send(); }
});
```

- Tooling: `npm i express bcrypt`, `nodemon` / `node --watch server.js` (`"dev"` script). **REST Client** (VS Code): `request.rest`, requests separated by ###, click "Send Request" — test API without a frontend.

Remember: cookie = ID only. `httpOnly` blocks JS, `secure` = HTTPS, `sameSite` = cross-site rule. Hash ≠ encrypt. 401 = who are you? 403 = not allowed.

5 · Tools used

Overview

Tool	What it is	Topic
Node.js + npm	run server-side JS / manage packages	all
VS Code	editor (`code .`)	all
Jest	unit testing framework	Jest
curl	HTTP requests from CLI	API
Postman	GUI API client / tester	API
Chrome DevTools	Network, Cookies, Console	API, Cookies
/docs	interactive API documentation	API
Express, EJS	web framework / template engine	MVC, Cookies
mysql2, dotenv, uuid	MySQL client / .env / IDs	MVC
Docker Compose	runs the lab DB	MVC
cors, cookie-parser, crypto	cross-origin / read cookies / UUID	Cookies
bcrypt	password hashing	Cookies
REST Client, Live Server	VS Code extensions	Cookies
nodemon / node --watch	auto-restart server	Cookies

npm

Command	Meaning
`npm init -y`	create package.json with defaults
`npm i pkg`	install dependency into node_modules
`npm i -D pkg` / `--save-dev`	devDependency (dev/test only)
`npm test` / `npm run dev` / `npm start`	run package.json scripts

Jest extras [+]

Item	Meaning
`jest --watch`	rerun only tests related to changed files
`jest --watchAll`	rerun all tests on any change
`jest --coverage`	collect & report test coverage
`jest --verbose`	show each test with suite hierarchy
`beforeEach/afterEach`	run before/after every test
`beforeAll/afterAll`	run once before/after all
`test.only`	run just this test temporarily

curl [+]

`-H` header (slides) · `-X POST` method · `-d '{..}'` body · `-i` show headers/status · `-u user:pass` Basic Auth

Postman

- Lab uses `web.postman.co` (no install). Method + URL → tabs **Params**, **Authorization** (Basic / Bearer), **Headers**, **Body** (raw JSON) → **Send**. Response: status, time, headers, body.
- **Collections**, variables like `{{base_url}}` [+]

Test scripts [+]: written in **Scripts > Post-response** (older versions: Tests tab), using `pm.test` with `pm.response.to.have.status()` or `pm.expect`.

```
pm.test("Status code is 200", () => {
  pm.response.to.have.status(200);
});
pm.test("Successful POST", () => {
  pm.expect(pm.response.code).to.be.oneOf([201, 202]);
});
pm.test("has userId", () => {

pm.expect(pm.response.json()).to.have.property("userId");
});
```

Chrome DevTools

Network: see requests (discover undocumented endpoints, copy request). **Application** → **Cookies**: view/edit/delete `sessionId`, `rack.session`, flags. **Console**: `document.cookie` (HttpOnly cookies hidden).

Express packages

Package	Key API
express	`Router()`, `express.json()`, `express.static()`, `res.json/send/status/sendStatus/render/cookie/clearCookie`
mysql2 / dotenv	`createConnection`, `db.query(sql,[p],cb)` / `dotenv.config()` → `process.env`
uuid / crypto	`uuidv4()` / `crypto.randomUUID()`
cors / cookie-parser	`cors({origin, credentials:true})` / `req.cookies`
bcrypt	`genSalt`, `hash`, `compare` (Promises → await)

VS Code extensions & misc

- **REST Client**: `request.rest`, separate with `###`, click **Send Request**; blank line between headers and body [+].
- **Live Server**: right-click index.html → Open with Live Server → 127.0.0.1:5500.
- **nodemon** vs `node --watch`: both auto-restart; slides install nodemon but the dev script uses `node --watch`.
- **Docker Compose**: `docker-compose.yaml` runs MySQL; `docker compose up -d` [+].
- **Practice sites**: icanhazdadjoke, randomuser.me, pokeapi, httpbin, dog.ceo, api.mathjs.org, todo-list-testing (`make run-dev`), base64decode.org, the-internet.herokuapp.com, semver.org.

6 · Beyond the slides (worth knowing)

Whole page = [+] not in slides, directly related

1. req.params vs req.query vs req.body

Source	Request	Express
params = path segment	`GET /products/5`, route `/products/:id`	`req.params.id` → "5"
query = after ?	`GET /tasks?status=In%20Progress`	`req.query.status`
body = POST/PUT data	`POST /users` + JSON	`req.body.name` (needs `express.json()`)

params/query are always **strings**. Typical use: params = which item, query = filter/search, body = data to create/update.

2. XSS vs CSRF

	XSS	CSRF
What	attacker's **script runs on your page**	another site makes the **victim's browser send a request** to a site they're logged into (cookies attached automatically)
Defence here	EJS `<%= %>` escaping; `httpOnly` cookies	`sameSite` lax/strict; CSRF tokens

3. Async tests in Jest

```
test("hash matches", async () => {
  const hash = await bcrypt.hash("pw", 10);
  expect(await bcrypt.compare("pw", hash)).toBe(true);
});
await expect(promise).resolves.toBe(true);
await expect(promise).rejects.toThrow("Not found");
```

Without `await`/`return` the test can finish before the Promise and pass wrongly.

4. PUT vs PATCH

	PUT	PATCH
Meaning	replace whole resource	update only sent fields
Missing fields	wiped/reset	kept

Lab link: PUT without email → email deleted. Consistent with "replace", but the slides call it a **Bug**: the API should validate (400) or offer PATCH. In the exam, follow the slides.